

```
In [1]: print("Hello World")
```

Hello World

```
In [2]: print("Hello World")
```

Hello World

H1

h2

h3

h4

h5

h6

This is a paragraph

1. Comments:- are the non-executable piece of code that is used to increase the readability of our program.

1. Single-line comment:- represented with '#'.
2. Multi-line comment:- represented with '''....''' or """....."""

```
In [6]: # This is a single line comment
```

```
In [8]: # """this
# is
# a
# multi-line
# comment"""
```

2. Keywords :- Reversed or pre-defined words present in our python library.

Python have 35+ keywords.

```
In [9]: help('keywords')
```

Here is a list of the Python keywords. Enter any keyword to get more help.

False	class	from	or
None	continue	global	pass
True	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	for	not	

```
In [ ]: finally # green and bold words are keywords
print() # green and words having () are predefined functions
var # words in black are variables
'red', "qwerty", "'wertyuio'", ""23456789"" # red and words having quotations
# are valid string.
```

3. Identifiers:- are the names that we give inorder to identify any variable, class, function etc.

While naming an Identifier we must follow "Identifier's Rule".

```
In [10]: # Rule 1: cannot use any keywords.
else = 12
```

```
Cell In[10], line 2
    else = 12
    ^
SyntaxError: invalid syntax
```

```
In [11]: # Rule 2: cannot start an Identifier's name with digits.
12var = 'apple'
```

```
Cell In[11], line 2
    12var = 'apple'
    ^
SyntaxError: invalid decimal literal
```

```
In [12]: var12 = 'apple'
```

```
In [13]: # Rule 3: cannot use spaces in between Identifier's name.
var one = 12345678
```

```
Cell In[13], line 2
    var one = 12345678
    ^
SyntaxError: invalid syntax
```

```
In [14]: # Rule 4: cannot use any special characters except underscore '_'.
var$1 = 1234567
```

Cell In[14], line 2

```
var$1 = 1234567
```

^

SyntaxError: invalid syntax

```
In [15]: var_1 = 1234567890
```

```
In [16]: # Rule 5: Python is a case-sensitive language i.e: 'A' != 'a'
a = 'apple'
print(A)
```

NameError Traceback (most recent call last)

Cell In[16], line 3

```
1 # Rule 5: Python is a case-sensitive language i.e: 'A' != 'a'
```

```
2 a = 'apple'
```

```
----> 3 print(A)
```

NameError: name 'A' is not defined

4. Variables:- are the container that helps us to store values in it.

It is also used for memory allocation.

```
In [17]: var = 12345
```

```
In [18]: var
```

```
Out[18]: 12345
```

5. Datatypes:- are used to represent the kind of data we are dealing with.

```
In [19]: # Primary Datatype:-
# 1. Numeric Datatype:-
# int:
var1 = 123456789
type(var1)
```

```
Out[19]: int
```

```
In [20]: # float:-
var2 = 234.000
type(var2)
```

Out[20]: float

```
In [23]: # complex:-  
var3 = 64567+99j  
type(var3)
```

Out[23]: complex

```
In [24]: # 2. Non-Numeric Datatype:-  
# bool:-  
var4 = True  
type(var4)
```

Out[24]: bool

```
In [25]: # string:-  
var5 = 'qweertyui'  
type(var5)
```

Out[25]: str

```
In [26]: # Sequential Datatype:-  
# 1. List:- are collection of individual elements separated by , and enclosed  
# within [].  
a = [12,2,True,'star',78.99,67+9j]  
type(a)
```

Out[26]: list

```
In [27]: # 2. Tuple:- are collection of single elements separated by , and enclosed  
# within ().  
b = (12,2,True,'star',78.99,67+9j)  
type(b)
```

Out[27]: tuple

```
In [28]: # 3. Set:- are collection of single items separated by , and enclosed within {}.  
c = {12,2,True,'star',78.99,67+9j}  
type(c)
```

Out[28]: set

```
In [30]: # 4. Dictionary:- are collection of paired elements (key:value) enclosed within  
# {}.  
d = {'1':'apple','b':'bat',False:78.99,56+8j:78+8j}  
type(d)
```

Out[30]: dict

Input/ Output in Python:

input() : a predefined function that allows user to give their values.

```
In [33]: name = input("Enter your full name: ")
```

```
In [34]: name
```

```
Out[34]: 'wertuio'
```

```
In [36]: a = input("Enter 1st number: ")
b = input("Enter 2nd number: ")
c = a+b
print(c)
print(type(c))
```

```
4578
<class 'str'>
```

Typecasting: is a process of converting one's default datatype to another datatype explicitly.

It is also known Datatype Conversion

```
In [37]: a = int(input("Enter 1st number: "))
b = int(input("Enter 2nd number: "))
c = a+b
print(c)
print(type(c))
```

```
141
<class 'int'>
```

```
In [38]: x = [2,4,6,'apple']
type(x)
```

```
Out[38]: list
```

```
In [39]: x1 = set(x)
type(x1)
```

```
Out[39]: set
```

```
In [ ]:
```